

CMPU 4021 DISTRIBUTED SYSTEMS

Student Name: Jana Sy | Student ID: C22363431 | Date: 20/11/25

Declaration: "I declare that this work, which is submitted as part of my coursework, is entirely my own, except where clearly and explicitly stated."

- Jana Sy

1. Application Architecture

For my distributed system application, it simulates a food marketplace with seller nodes which have an inventory of items that they offer for sale, and multiple buyer nodes that can connect to these sellers to purchase items currently on sale, aligning with the requirement, '*at least one seller and multiple buyers must be present*'.

The seller is essentially a TCP server that:

- Rotates which item is on sale depending on the time and quantity of stock.
- Limits each sale to a maximum of 60 seconds, but their selling time can vary.
- Decrements the stock of the item for sale when buyers make a purchase.
- Broadcasts real-time updates to all buyers whether other buyers buy some stock of the item for sale, time left for sale, and the stock left after a buyer made a purchase.

The buyers connect as TCP clients to the server which they:

- Can join and leave the market whenever they please.
- List items on the seller's inventory as well which item is on sale.
- Attempt purchases.
- Receive live notifications from the seller.

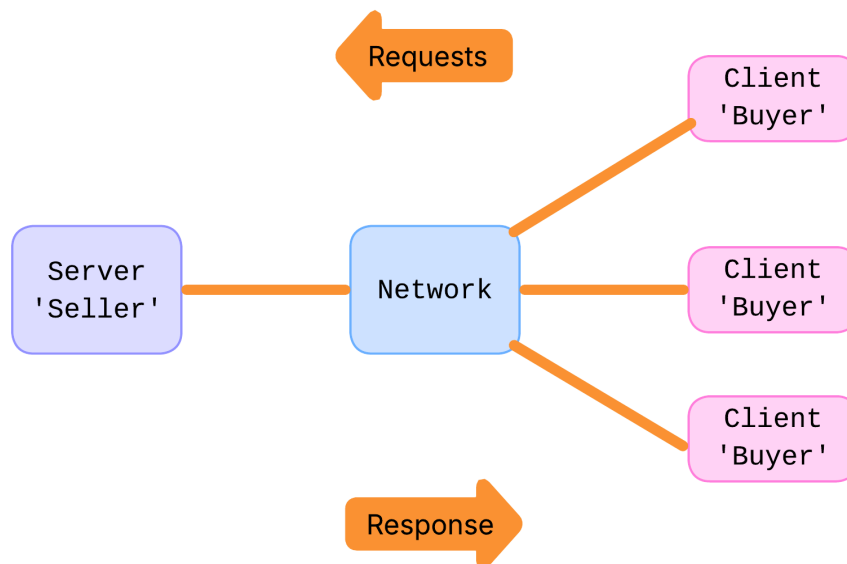
1.2 Architectural Model

This implementation of my application uses a Client-Server Model¹ with multiple servers 'sellers' and multiple clients 'buyers' using configurable ports, a distributed architecture where clients request services and servers provide them.

¹GeeksforGeeks, "ClientServer Model," *GeeksforGeeks*, Aug. 27, 2025. <https://www.geeksforgeeks.org/system-design/client-server-model/>

Client: any device or software that initiates communication by requesting data or services from a server.

Server: a powerful system that listens for and responds to client requests by delivering data or performing tasks. Servers often handle multiple simultaneous client requests.



Simple Client-Server Model.

For my application, it essentially has separate TCP server processes ‘sellers’ that run parallel from one another, and act as a central authority which manages the inventory, sale-timing, and broadcasts. In which clients ‘buyers’ connect to one server ‘seller’ at a time to request information ‘make purchases’.

- Each seller is a separate TCP server process.
- Each buyer is a separate client process talking to one seller over sockets.
- Sellers run in parallel, handle multiple buyers concurrently via threads.
- All shared state, such as: inventory, current sale item, and timers are all inside the seller program, and all connected buyers only see it via messages.

1.3 Components

Seller node

My seller node is implemented as a TCP server running on a configurable port.

Its job is to manage their own ‘shop’ and coordinate all connected buyers. Each seller maintains the inventory, the current item for sale, and the sale timer which

determines how long an item is up, and it handles buyer purchases concurrently using threads.

- The seller creates a socket, allowing multiple incoming buyer connections concurrently.

```
# For a TCP* socket, AF_INET = internet address family for IPV4, SOCK_STREAM = socket type for TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_address = (host, port)

#binds the socket to the address above
print(f"Starting TCP Connection for {s} %s port %s" % server_address)
server.bind(server_address)

new_sale()
timer_thread = threading.Thread(target=sale_timer_loop, daemon=True)
timer_thread.start()

# enables a server to accept connections
server.listen()
print(f"{s} listening for connections...")
```

- This allows it so that the seller can accept multiple incoming buyer connections concurrently.
- Every time a buyer connects, they have their own thread, allowing the seller to process multiple buyers at once.

```
while True:
    conn, addr = server.accept()
    print(f"{s} accepted connection by {addr}")

    # creates a new thread,
    # target=handle_client -> what function the thread runs
    # args=(conn, addr) -> passes the arguments to the target fns
    # daemon=True -> background thread that doesnt stop the program from exiting
    client_thread = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    #starts the actual thread, if .start() isn't called, the thread exists but isn't run.
    client_thread.start()
```

- Since the seller has several shared variables: inventory, current_item, sale_end_time, and buyers lists. These are modified by multiple buyer threads, so a threading.Lock() is used to prevent race conditions
- This ensures correct stock updates even if multiple buyers buy at the same time.
- A timer thread handles the sale duration: checking every second whether the current sale has expired, ends the sale when time runs out, and starts a new sale automatically.
- The seller also chooses a new item at random and sets a timer between 10-60 seconds.

- To process buyer commands, a separate `handle_client()` function is used for the seller to respond with: “inventory” prints the full item inventory, “sale” shows the current item on sale with the time left, “buy <ITEM> <AMOUNT> -> to make a purchase, and a simple “commands” to display a list of useable commands for the buyer.
- When a buyer sends a “buy” request, the seller parses the item and the amount. It gets validated if the:
 - Item exists
 - Amount is > 0
 - Amount <= stock
 - Item matches current item on sale
- It then deducts the stock safely under a lock, sending a confirmation to only that buyer, and broadcasting the remaining stock to all buyers.
- The seller also has a `broadcast()` function to send updates to all connected buyers, each message ending with `\n` to ensure each reply is displayed separately.

Buyer Node

My buyer node is implemented as a TCP client which connects to the “seller” server and uses a command-line interface to interact with their shop.

- The buyer enters their unique name and the seller’s port number, their name acting as their unique ID.

```
buyer = input("Enter your name: ")
b = f"{buyer}"

port_num = input("Enter port for this shop (e.g 5000): ").strip()
if not port_num:
    port_num = "5000"

port = int(port_num)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server_address = (host,port)
client.connect(server_address)
```

- Once a buyer is connected, the buyer starts a background receiver thread, this thread constantly listens for messages from the seller, such as:
 - Sale announcements
 - Purchase broadcasts
 - Stock updates
 - New sale notifications
 - Server shutdowns

- Viable commands are directly sent to the seller, commands like: “inventory”, “buy <ITEM> <STOCK>”, etc. and for a buyer to leave that seller’s shop, simply typing “quit” or “exit” terminates the connection.

2. Running The Application

The source code for both “buyer” and “seller” are each contained in their respective .py files: buyer.py, and [seller.py](#). To keep the code simple and direct, the seller identifies their own port and unique ID. Prompting the user to enter which unique seller name they would like to use for their seller/shop, and which port they would like to use (default is 5000).

This same execution is applied to the buyer as well, the user will be prompted to enter a unique buyer name and which port they would like to join (default is 5000).

Firstly, to execute [seller.py](#):

- CD into the correct folder using the terminal, where the python file/s are stored (in my case it’s: C22363431-DS-CMPU4021-ASSIGNMENT)

```
Anhre Distributed Systems 03:42 21ms cd .\C22363431-DS-CMPU4021-ASSIGNMENT\
Anhre C22363431-DS-CMPU4021-ASSIGNMENT (Creating-the-shop) 03:42 18ms
```

- Execute the program in the terminal using: python [seller.py](#)
- Enter your unique seller name and which port you would like to connect to:

```
Anhre C22363431-DS-CMPU4021-ASSIGNMENT (Creating-the-shop) 03:42 18ms python seller.py
Enter your name, seller: X
Enter port for this shop (e.g 5000):
Starting TCP Connection for [X] localhost port 5000
[X]: NEW SALE -> OIL for 57s.
[X] listening for connections...
```

Secondly, to execute [buyer.py](#), make sure to have a seller up to join their respective port:

- Cd into the correct folder using the terminal, where the python file/s are stored (in my case it’s: C22363431-DS-CMPU4021-ASSIGNMENT)

```
Anhre Distributed Systems 03:42 21ms cd .\C22363431-DS-CMPU4021-ASSIGNMENT\
Anhre C22363431-DS-CMPU4021-ASSIGNMENT (Creating-the-shop) 03:42 18ms
```

- Execute the program in the terminal using: python [buyer.py](#)
- Enter your unique buyer name and type a port in which a seller is already on:

```
Anhre C22363431-DS-CMPU4021-ASSIGNMENT (Creating-the-shop) 03:48 102ms python buyer.py
Enter your name: Y
Enter port for this shop (e.g 5000):
Y, you have connected to shop!
```

3. Alternative System Design

In the implemented system, the market was built using a client-server architecture: in which single seller nodes run parallel from one another and each acts as a central TCP server, and buyers connect as clients to whichever server they want using the seller’s respective ports.

An alternative design solution could have the market be built as a peer-to-peer (P2P) system², where participants share resources directly with each other without the need for a centralised server, where:

- Any node can act as a buyer, a seller, or both.
- Sellers announce items for sale by multicast.
- Buyers discover sellers dynamically and connect to them directly over TCP to make purchases.

This architecture is more scalable, but more complex to implement, thus it was not chosen for the final system implementation.

3.1 Alternative Components

In this alternative design, each running instance would be a Market Node with the following logic:

- Market node would maintain a unique nodeID, role (buyer/seller), and local inventory if it's acting as a seller.
- A multicast announcer on the seller's side:
 - Sending UDP multicast messages.
 - Running periodically to advertise active sale items.
- A multicast listener on the buyer's side:
 - Subscribes to a multicast group.
 - Collecting and maintaining a list of active sellers and their current items.
- For handling purchases, sellers would listen on TCP ports to accept purchase requests. Buyers will connect to that TCP to send buy requests.
- Similar to the implemented design, each seller node will use threads and a lock which:
 - Handle multiple buying nodes concurrently.
 - Safely updating inventory.
 - Managing time-limited sales.

The main difference between this and the implemented solution is the decentralised discovery, and multiple sellers, rather than one central seller server.

3.2 Component Communication

Seller Node

The seller node would start a TCP server which accepts purchase requests, periodically sending multicasts, "I'm selling [X]." announcements with their:

- Seller nodeID.

²GeeksforGeeks, "PeertoPeer (P2P) Architecture," *GeeksforGeeks*, Jul. 23, 2025. <https://www.geeksforgeeks.org/system-design/peer-to-peer-p2p-architecture/>

- Current item in stock.
- Remaining stock of that item.
- TCP port for the buyer to connect to.

When a buyer connects via TCP and sends a purchase request, the seller will:

- Check for stock and time limit.
- Then update their inventory.
- Reply with either success or a failure.

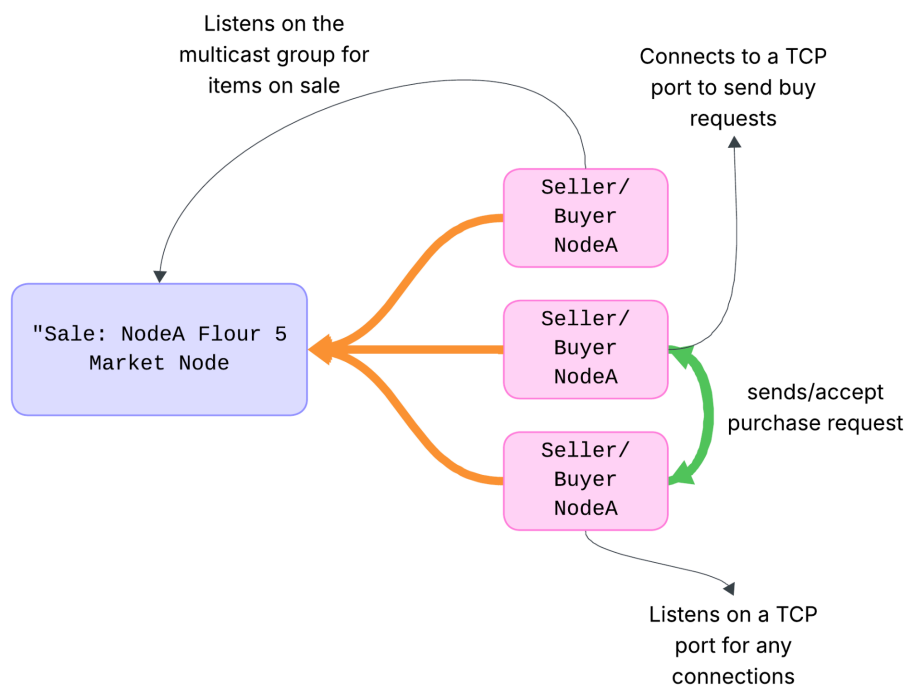
Buyer Node

The Buyer node would also listen on the multicast group for any sellers on the multicast:

- The user will choose which seller/item they want to buy from.
- The buyer opens a TCP connection directly to the selected seller to send a “buy” request.
- They either receive a confirmation or failure of purchase.

In case of failure

If a seller goes offline, their multicast announcements for sale will stop. Buyers on the other hand will eventually remove that seller from their local view after a timeout without breaking the entire system.



Alternative P2P Multicast System

3.3 Why was it not implemented?

Although this peer-to-peer (P2P), multicast-based design is more flexible and closer to a real marketplace. It has a higher system complexity, requiring each node to:

- Listen to multicast.
- Send multicast
- Handle TCP purchase requests
- Maintain a dynamic table of active sellers
- Handle node join/leave detection

There may also be more failure scenarios, if a seller crashes, a buyer/s must detect this by noticing missing multicast announcements. Or, if a network drops multicast packets, buyers remove sellers incorrectly.